Chalmers | Göteborgs Universitet

Concurrent Programming TDA384/DIT392

21 August 2025

Exam supervisor: N. Piterman (piterman@chalmers.se, 031 772 6053)

(Exam set by N. Piterman and G. Schneider, based on the courses given in Oct-Nov 2024 and Jan-Mar 2025)

Material permitted during the exam (hjälpmedel):
Two textbooks; four sheets of A4 paper with notes (single or double-sided); English dictionary (no smart phones allowed).

Grading: You can score a maximum of 70 points. Exam grades are: between 28–41 (3), between 42–55 (4), 56 or more (5).

Passing the course requires passing the exam and passing the labs. The overall grade for the course is determined as follows: between 40–59 (3), between 60-79 (4), 80 or more (5).

The exam **results** will be available in Ladok within 15 *working* days after the exam's date.

Instructions and rules:

- Please write your answers clearly and legibly: unnecessarily complicated solutions will lose points, and answers that cannot be read will receive no points!
- Justify your answers, and clearly state any assumptions that your solutions may depend on for correctness.
- Answer each question on a new page. Glance through the whole paper first; four questions, numbered Q1 through Q4. Do not spend more time on any question or part than justified by the points it carries.
- Be precise. In your answers, try to use the programming notation and syntax used in the questions. You can also use pseudo-code, *provided* the meaning is precise and clear. If need be, explain your notation.

This page is left empty intentionally.

Q1 (13p). You are going to construct the transition table of the pseudo-code below and answer few questions about its properties.

int go= 1; **int** flag1= 1; **int** flag2= 1;			
	p		q
	while(true) {		**while**(true) {
p_1	*//NCS (non-critical section)*	q_1	*//NCS (non-critical section)*
p_2:	flag1= 0;	q_2:	flag2= 0;
p_3:	go= 1;	q_3:	go= 0;
p_4:	**while**(flag2!= go) { };	q_4:	**while**(flag1== go) { };
p_5	*//CS (critical section)*	q_5	*//CS (critical section)*
p_6:	flag1= 1;	q_6:	flag2= 1;
	}		}

Preliminaries. A full state is of the form $(p_i, q_j, \texttt{flag1}, \texttt{flag2}, \texttt{go})$, where i and j range over $\{2, 3, 4, 6\}$, and flag1, flag2, and go range over 0 and 1. We will consider a a *reduced state* of the form $(p_i, q_j, \texttt{go})$ (abstracting away flag1 and flag2). As transitions into p_4 and q_4 set the value of go, we can ignore the value of go when both p and q are in locations 2 or 3. Only 16 states are reachable.

Notation: We denote the value of go by x when we do not care about it. For example, (p_2, q_2, x) corresponds to either $(p_2, q_2, 0)$ or $(p_2, q_2, 1)$.

Here is a partial state transition table for the program above. As mentioned, only 16 states are reachable from the initial state $(p_2, q_2, 1)$.

state		new state if p moves	new state if q moves
s1	$(2, 2, x)$		$(2, 3, x)$ = s2
s2	$(2, 3, x)$		
s3	$(3, 2, x)$		
s4	$(3, 3, x)$		$(3, 4, 0)$ = s15
s5	$(2, 4, 0)$		$(2, 6, 0)$ = s6
s6	$(2, 6, 0)$	$(3, 6, 0)$ = s6	
s7	$(4, 2, 1)$		$(4, 3, 1)$ = s9
s8	$(6, 2, 1)$		
s9	$(4, 3, 1)$		
s10	$(4, 4, 1)$	no move (s10)	
s11	$(4, 4, 0)$	$(6, 4, 0)$ = s14	
s12	$(4, 6, 1)$		$(4, 2, 1)$ = s7
s13	$(6, 3, 1)$		
s14	$(6, 4, 0)$		
s15	$(3, 4, 0)$		no move (s15)
s16	$(3, 6, 0)$	$(4, 6, 1)$ = s12	

(Part a) Fill in the blank entries in the table. *(5p)*

(Part b) Does the protocol satisfy mutual exclusion? Explain. *(3p)*

(Part c) Assuming a fair scheduling: is the protocol starvation-free? Explain. *(3p)*

(Part d) Will the program still satisfy mutual exclusion and be starvation-free if we replace all occurrences of `flag1` by `flag2`, and vice-versa (all occurrences of `flag2` by `flag1`)? Explain. *(2p)*

Answer:

(Part a)

state		*new state if p moves*	*new state if q moves*
s1	$(2, 2, x)$	$(3, 2, x)$ = *s3*	$(2, 3, x)$ = *s2*
s2	$(2, 3, x)$	$(3, 3, x)$ = *s4*	$(2, 4, 0)$ = *s5*
s3	$(3, 2, x)$	$(4, 2, 1)$ = *s7*	$(3, 3, x)$ = *s4*
s4	$(3, 3, x)$	$(4, 3, 1)$ = *s9*	$(3, 4, 0)$ = *s15*
s5	$(2, 4, 0)$	$(3, 4, 0)$ = *s15*	$(2, 6, 0)$ = *s6*
s6	$(2, 6, 0)$	$(3, 6, 0)$ = *s6*	$(2, 2, x)$ = *s1*
s7	$(4, 2, 1)$	$(6, 2, 1)$ = *s8*	$(4, 3, 1)$ = *s9*
s8	$(6, 2, 1)$	$(2, 2, x)$ = *s1*	$(6, 3, 1)$ = *s13*
s9	$(4, 3, 1)$	*no move (s9)*	$(4, 4, 0)$ = *s11*
s10	$(4, 4, 1)$	*no move (s10)*	$(4, 6, 1)$ = *s12*
s11	$(4, 4, 0)$	$(6, 4, 0)$ = *s14*	*no move (s11)*
s12	$(4, 6, 1)$	*no move (s12)*	$(4, 2, 1)$ = *s7*
s13	$(6, 3, 1)$	$(2, 3, x)$ = *s2*	$(6, 4, 0)$ = *s14*
s14	$(6, 4, 0)$	$(2, 4, 0)$ = *s5*	*no move (s14)*
s15	$(3, 4, 0)$	$(4, 4, 1)$ = *s10*	*no move (s15)*
s16	$(3, 6, 0)$	$(4, 6, 1)$ = *s12*	$(3, 2, x)$ = *s3*

(Part b) *Yes. States* $(6, 6, 1)$ *and* $(6, 6, 0)$ *are not reachable.*

(Part c) *It is starvation-free. From* $s4$ *there is a choice whether to go to* $s9$ *or* $s15$*. If going for* $s9$*, then there is no choice but to continue to* $s11, s14, s5$ *now we have to show that q will enter the critical section. This is if either we go directly to* $s6$ *or* $s15, s10, s12$*. The case of going from* $s4$ *to* $s15$ *is dual.*

(Part d) *Yes, since changing the names in all occurrences will not change the behaviour of the program given the way the flags are used.*

Q2 (12p). Consider this two-threaded program with threads `s` and `t`. The two threads share the variables `n1` and `n2`. The function *incognito*$(\cdot,\cdot)$ is an unknown function that gets two integer parameters and returns one integer. The function *incognito* is meant to remain completely unknown (the only thing we know about the function is that it always terminates and that it returns a numerical value).

int c1 = 0; c2 = 1			
	s		t
s_1:	**while**(c1 < 500) {	t_1:	**while**(c2<=500) {
s_2:	c1 = c1 + 1;	t_2:	c2 = c2 + 1;
s_3:	c2 = incognito(c1,c2)	t_3:	c1 = incognito(c1,c2)
	}		}

The labels s_1, s_2, s_3, t_1, t_2 and t_3 are given only for ease of reference.

(Part a) Are there race conditions? If so give an example, otherwise state why it is not the case. *(2p)*

(Part b) Can the program run forever? If so, give an an example of an infinite execution. (Note: you can make any assumptions you want about the function *incognito*$(\cdot,\cdot)$ knowing that it always terminate and gives a value.) *(2p)*

(Part c) Show an execution in which the program terminates. *(2p)*

(Part d) Does the program terminate in all *fair* executions? *(2p)*

(Part e) Let us assume that the function *incognito*$(\cdot,\cdot)$ is *strictly increasing monotonic* in its parameters, that it is, it always gives a greater number than both its parameters. (Example: *incognito*$(45, 430)$ gives a value that is greater than 430.). Under this assumption: does the program always terminate? Justify your answer. *(2p)*

(Part f) Let us assume now that the function *incognito*$(\cdot,\cdot)$ is *strictly decreasing monotonic* in its parameters, that it is, it always gives a smaller number than both its parameters. (Example: *incognito*$(45, 430)$ gives a value that is smaller than 45.). Under this assumption: does the program always terminate? Justify your answer. *(2p)*

Answers:

(Part a) *Yes, executing t first till termination and then s will give a different result than alternating their execution (also, it is possible to get an infinite execution as show in Part b).*

(Part b) *A possible infinite execution would be one where t and s alternate executions and the incognito function always returns 1 (so the c1 and c2 counters are always reset and they loop forever).*

(Part c) *Process t runs until c2 is 500 and terminates. Then if c1 > 500 process s terminates as well. Otherwise, the only possible change*

to $c1$ *is s increasing it. Hence,* $c1$ *will eventually increase above* 500.

(Part d) *No. See the scenario given as answer to Part a.*

(Part e) *Yes, it always terminates as at the end of each iteration of the loops the values of* $c1$ *and* $c2$ *will be increasing and will eventually reach 500.*

(Part f) *No, the program might not terminate as the incognito function might return a negative value and thus the loops will never terminate when the threads take turn in executing.*

Q3 (15p). In what follows you have 5 subquestions (Parts a to e) concerning different topics seen in the course. Each part is a multiple-choice question. The grading for each question is as follows:

- For a right answer you will get 3 points;
- If you do not answer the question, you will get 0 points;
- If you answer wrongly, you will get -1 (a negative point).

The total amount of points will be done summing all the points for each part. So, if you answer correctly Part a, incorrectly Part b, and do not answer any of the rest, you will get 2 points (3 points for Part a minus 1 point for Part b, and 0 for the rest). Note that no negative points for the whole question Q3 will be given (the minimum number of points you may get for the whole question is 0). That is, if you do answer wrongly in each part, you will not get -5 but 0.

(Part a). (3p)

According to Amdahl's law, if the fraction p of a program can be parallelised, then, the maximum speedup that can be achieved by n processors is $\frac{1}{(1-p)+\frac{p}{n}}$. You have a program where 50% of the program must be done sequentially and 50% of the program can be parallelised. What is the maximum speedup that you can achieve given unlimited resources (i.e., increase the number of processes as you wish).

a) The speedup can never be more than double.

b) With 1000000 processors, the program can run 3 times faster.

c) With at least 1000 processors the program can run more than 2 times faster.

d) The program cannot run more than 1.5 times faster.

(Part b). (3p)

In what follows we state few properties about the pseudocode of the program shown in Fig. 1. Which one is true?

a) It satisfies mutual exclusion, deadlock-freedom and starvation-freedom.

b) It doesn't satisfy mutual exclusion but it is deadlock-free and starvation-free.

c) It does satisfy mutual exclusion and deadlock-freedom, but it is not starvation-free.

d) It doesn't satisfy mutual exclusion, deadlock-freedom nor starvation-freedom.

```
Semaphore sem = new Semaphore(1); int count
```

thread t		thread u	
`int c0,c1;`		`int c0 = 0;`	
1	`sem.down();`	`sem.up();`	6
2	`c0 = count - 1;`	`count = c0 + 10;`	7
3	`c1 = c0 * 2;`	`c0 = count + 2;`	8
4	`count = c0 + c1 + 2;`	`sem.down();`	9
5	`sem.up();`		

Figure 1: Q1-2: Pseudocode of program `someCounting`.

e) The whole question is irrelevant as the two threads cannot be interleaved.

(Part c). (3p)

The following code shows part of a telephone system implemented in Erlang:

```
ringing(A, B) ->
       receive
            {B, answered} ->
                 A ! {stop_tone, ring},
                 switch ! {connect, A, B},
                 conversation(A, B);
            {A, on_hook} ->
                 A ! {stop_tone, ring},
                 B ! finish,
                 idle()
       end.
```

The provided Erlang program defines the function `ringing/2` that handles the ringing phase of a telephone call between two parties, represented by processes A and B. In what follows you will see 5 assertions about the program. Which one is NOT correct?

a) If the received message matches the tuple `{A, on_hook}`, it means that process A has gone on-hook (hung up) before process B answered. In this case Process A is sent a message `{stop_tone, ring}` to inform it that the ringing tone should stop.

b) The function `ringing/2` gets stuck in case of receiving a message {A, finish}.

c) Both processes A and B get messages when a process is executing `ringing(A,B)` and receives a message `{A, on_hook}`.

d) In case of receiving a message {A, finish}, the function `ringing/2` ignores the message.

e) I can call the function `ringing/2` with parameters C and D (where C and D are processes).

(Part d). (3p)

Consider the following Erlang code:

```
start(Server) ->
  Server ! {do_work, self(), task1},
  timer:sleep(50),
  Server ! {do_work, self(), task2},

  receive
    {result, R1} -> io:format("First task: ~p~n", [R1])
  end,

  receive
    {result, R2} -> io:format("Second task: ~p~n", [R2])
  end.
```

The server process handles `{do_work, From, Task}` message by spawning a process that (a) will take care of it and (b) later reply to `From` with `{result, Answer}`.

Which of these properties is true:

a) The server may get the message with `task2` first.

b) The delay between sending `task1` and sending `task2` will ensure that the replies will arrive in order.

c) The replies may arrive out of order.

d) The second `receive` will always match the second reply, regardless of arrival order.

(Part e). (3p)

Consider the following Java class:

```
class Shared {
  final Object lock = new Object();
  int count = 0;

  public void increment() {
```

```
    synchronized (lock) {
      count++;
    }
  }

  public void reset() {
    synchronized (this) {
      count = 0;
    }
  }
}
```

Assume multiple threads are calling both increment() and reset() concurrently.

Which of the following best describes the properties of this class?

a) There is no issue — both methods use synchronization.

b) The `synchronized(this)` in `reset()` protects the same data as `lock`, so mutual exclusion is ensured.

c) Synchronizing on `this` is invalid if another object uses a separate lock.

d) The two methods synchronize on different objects, so a race condition on `count` can occur.

Answers:

(Part a). *Option a: with a 50% parallelisation ($p = 0.5$) the formula is $\frac{2n}{n+1}$ so the value approaches to 2 as n increases but never reaches a speedup of 2 times.*

(Part b).

Option b. It is not mutual exclusive (e.g., if thread u calls `sem.up()` *first, then thread u can call* `sem.down()` *and both will be in the critical section). It is deadlock-free and starvation-free (it is never the case that both get stuck because they cannot execute* `sem.up()`*). The "right" code (to achieve mutual exclusion) is to exchange lines 6 and 9 (i.e., thread u should execute* `sem.down()` *at the beginning and* `sem.up()` *at the end).*

(Part c).

Option b: the function doesn't get stuck but just ignores the message.

(Part d).

Option c: We do not know the order in which the worker will handle the tasks. Although task1 is sent before task2, the random sleep in the worker means task2 may complete and send a reply first. Erlang preserves send order from one sender, but the worker is handling tasks and replying at unpredictable times, so replies may arrive in a different order.

(Part e). *Option d: Although both methods use synchronized, they do so on different objects: increment() locks on lock, while reset() locks on this. These do not block each other, so two threads can simultaneously modify count, causing a race condition.*

Q4 (16p). We present two solutions to a simplified version of the *river crossing* problem:

A boat crosses a river with exactly four passengers. The boat **must not** depart with any other number of passengers. One passenger in each group acts as the **captain**, initiating the crossing.

Below are two implementations of of this problem. Identify whether they are correct, explain the flaw if you find one, or the principles of the solution if there is no flaw.

(Part a). (8p) Analyze this monitor based solution:

```
class RiverA {
  private final Lock lock = new ReentrantLock();
  private final Condition canBoard = lock.newCondition();

  private int waiting = 0;
  private int onboard = 0;

  public void passenger() throws InterruptedException {
    boolean isCaptain = false;

    lock.lock();
    try {
      waiting++;
      while (waiting < 4) { canBoard.await(); }

      waiting--;
      onboard++;
      if (onboard == 4) {
        isCaptain = true;
        onboard = 0;
      }
      canBoard.signalAll();  // wake other threads
    } finally {
      lock.unlock();
    }

    System.out.println("Boarded.");

    if (isCaptain) {
      System.out.println("Boat is crossing...");
    }
  }
}
```

(Part b). (8p) Analyze this semaphore and barrier based solution:

```
class RiverB {
  Semaphore passengers = new Semaphore(0);
  Semaphore mutex = new Semaphore(1);
  CyclicBarrier barrier = new CyclicBarrier(4);
  int passengerCount = 0;

  public void passenger() throws Exception {
    boolean isCaptain = false;
    mutex.acquire();
    passengerCount++;
    if (passengerCount == 4) {
      releasePassengers(4);
      passengerCount -= 4;
      isCaptain = true;
    }
    mutex.release();
    passengers.acquire();
    System.out.println(role + " boarded.");
    barrier.await();
    if (isCaptain) {
      System.out.println("Boat is crossing...");
    }
  }

  private void releasePassengers(int n) {
    for (int i = 0; i < n; i++) s.release();
  }
}
```

Answers:

Part a) The solution is incorrect. When the fourth thread arrives it reduces the number of waiting back to 3 and signals to all. But then other threads that were not waiting could get the lock before those who were waiting. As a result we could get a boat with 4 captains.

Part b) The solution is incorrect. It starts by releasing 4 permits for 4 passengers only when passengers are ready. However, after releasing the mutex, the "captain" thread may be delayed. Other threads may arrive and proceed before the real captain reaches barrier.await(). As a result a boat can depart without a captain.

Q5 (14p). Consider the following Erlang code attempting to keep two counters in sync (functions `increment2` and `get_value2` are omitted as they just replace 1 with 2).

```
-module(counter).
-export([start/0, increment1/0, increment2/0, get_value1/0,
get_value2/0]).

start() ->
  Pid1 = spawn(?MODULE, loop_waiting, []),
  Pid2 = spawn(?MODULE, loop, [0, Pid1]),
  Pid1 ! {peer, Pid2},
  register(counter1, Pid1),
  register(counter2, Pid1),
  {Pid1, Pid2}.

increment1() ->
  counter1 ! {increment, self()},
  receive ok -> ok end.

get_value1() ->
  counter1 ! {get, self()},
  receive {value, V} -> V end.

loop_waiting() ->
  receive {peer, PeerPid} -> loop(0, PeerPid) end.

loop(Value, PeerPid) ->
  receive
   {increment, From} -> New = Value + 1,
                        PeerPid ! {sync, New},
                        From ! ok,
                        loop(New, PeerPid);

    {sync, NewValue} when NewValue > Value -> loop(NewValue, PeerPid);

    {sync, _} -> loop(Value, PeerPid);

    {get, From} -> From ! {value, Value},
                   loop(Value, PeerPid)
  end.
```

(Part a). (4p) Show a scenario where the synchronization of the two counters creates a problem.

(Part b). (10p) Change the program so that increments are never lost and so that every increment will eventually be updated in the entire system.

Answers:

Part a) Suppose the two counters start with 0. Two increments are sent to both, both increase their value to 1 and sent this message to the peer. Both peers ignore the update as it is not greater than their own value. The total number of increments was two but both counters settle on 1.

Part b) This can be done by counting separately the number of increments done to each counter separately. These are the functions that need updating:

```
% State: {MyIncr, PeerIncr}
start(Peer) ->
  Pid1 = spawn(?MODULE, loop_waiting, []),
  Pid2 = spawn(?MODULE, loop, [{0,0}, Pid1]),
  Pid1 ! {peer, Pid2},
  register(counter1, Pid1),
  register(counter2, Pid1),
  {Pid1, Pid2}.

loop_waiting() ->
  receive
    {peer, PeerPid} ->
      loop({0,0}, PeerPid)
  end.

loop({My, Peer}, PeerPid) ->
  receive
    {increment, From} ->
      NewMy = My + 1,
      PeerPid ! {sync, NewMy},
      From ! ok,
      loop({NewMy, Peer}, PeerPid);
    {sync, PeerMy} ->
      NewPeer = max(Peer, PeerMy),
      loop({My, NewPeer}, PeerPid);
    {get, From} ->
      From ! {value, My + Peer},
      loop({My, Peer}, PeerPid)
  end.
```